

Using Static Analysis Tools for Bug Detection in Object Relational Mappers

Natalie Dean¹ Erika Grandy¹ Hashir Jamil¹ Krishna Raju¹ Michael Sandrin¹ Zachary Ross¹

¹Department of Electrical Engineering & Computer Science, York University
{natrdean, egrandy, hashirj, krishnar, zachross, msandrin}@my.yorku.ca

Abstract

In this project, three cross platform static code analyzers are studied by examining their execution on open-source *Java* and *Python* object relational mapping (ORM) frameworks. The analysis tools: *SonarCloud*, *Coverity* and *Snyk* all exhibit unique idiosyncrasies that are investigated for deeper insight into the usage of these tools for *ORM frameworks*. All three tools gave varied results across the five frameworks tested. The tools' performances and the differences across the detected issues given by each is examined. Samples of reported issues are verified by human inspection. Finally, suggestions are made for using the three tools.

The full suite of supplementary figures, data and Jupyter notebook can be found at our [GitHub repository](#) [1].

1 Introduction

In the field of software development, ensuring the reliability and efficiency of code is crucial in complex systems and programs. *Object Relational Mappers* (ORMs) are an example of complex systems that act as a medium between *Object Oriented Programming* and databases. Systems such as *ORMs* can rely on data manipulation in a timely manner as well as they are susceptible to security issues, adding a potential for unwelcome data access and handling.

This report highlights the use of *static analysis tools* being used to detect bugs within *ORMs* as they are widely used in many applications. The tools were selected to show the use of static analysis within the software development space, while thoroughly underlining their capabilities, ease of use, effectiveness, and limitations.

Throughout the research and analysis displayed in this report, we provide valuable insight into how each tool is used and show how with the results given from each tool, *ORMs* can be improved on the software development side of the spectrum.

1.1 Static Analysis

Static code analysis is a technique that performs analysis of code without execution. Although static analysis is computationally intensive and is considered to be an undecidable problem [2]. This leads to approximate solutions when done at the large scale. Static analysis uses rules inferred by traversing abstract syntax trees, control flow graphs and call graphs to analyze the structure and behavior of the system [3, 4]. This can be seen in Fig. 1, a codebase can be analyzed after being built using tree/graph models of the code to sniff out anti-patterns that may be present. By applying predefined rule sets and standards to these representations, potential bugs such as unused variables, unreachable code, memory leaks, and unsafe coding practices can be discovered. The detected bugs are then reported to developers to either confirm and fix or dismiss as a false positive.

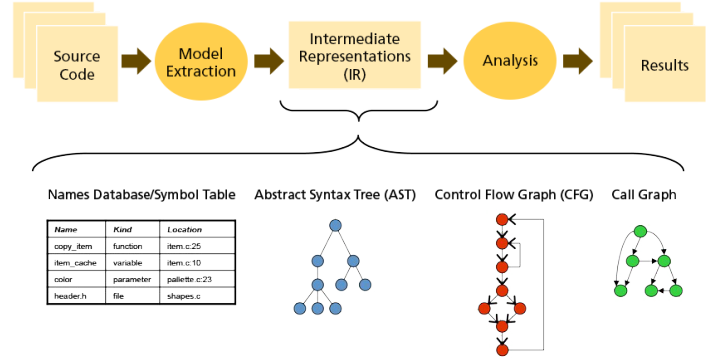


Fig. 1: Static Analysis Over Source Code Life Cycle [3]

When integrated into the software development workflow static code analysis provides continuous feedback and contributes significantly to enhancing code quality. In a generic continuous integration/continuous delivery (CI/CD) pipeline Fig. 2, these tools can be incorporated at either pre-build or post-build to offer extra layers of inspection [5]. In other words, these tools act either on the source code as written or a processed, intermediate representation of the code. The three tools studied here all analyze intermediate representations. Static code analysis is also inextricably linked with compiler design. There are many overlapping patterns found in both groups. Compilers use static analysis to optimize the code at runtime [4]. In essence, a code base is going through static analysis continuously throughout a development cycle. IDE and compiler tools do this in the background as the code is being written and compiled, while formal static analysis tools are used to do higher order quality assurances at downstream stages of CI/CD.

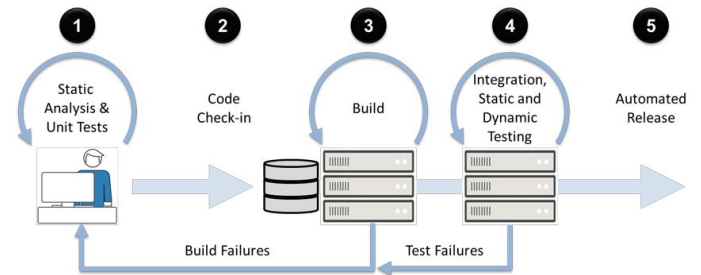


Fig. 2: Static Analysis In A Prototypical CI/CD Pipeline [5]

1.2 Object Relational Mappers

Two major programming paradigms are the object-oriented model that governs many languages and established architectural patterns

and the relational model that is found in database systems. Due to the nature of both models conflicting with one another, we get the famous object-relational "impedance mismatch" problem [6] as the two models are not mathematically equivalent views of the data and domain. The solution to this problem is the idea of object relational mapping, which can be thought of as a general process to allow both models to be incorporated and synchronized in a computing application. Mapping can be incorporated in a design as a general pattern and it can also be accomplished using a library/framework. This is where the systems under consideration in the study come into play. These five *ORM frameworks* are commonly used solutions for object-relational hybrid applications that make up the modern software architecture landscape.

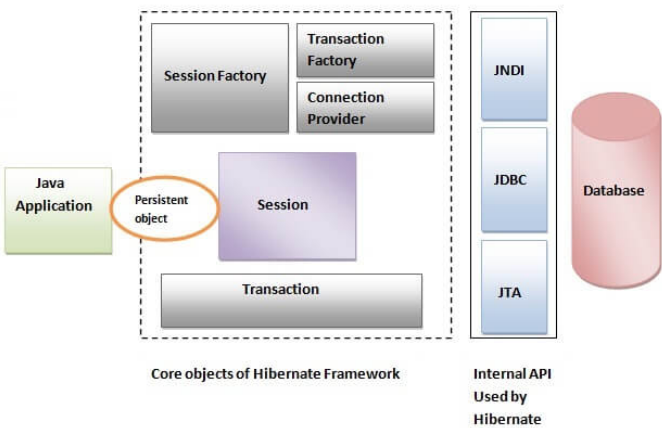


Fig. 4: ORM Reference Architecture using Hibernate [9]

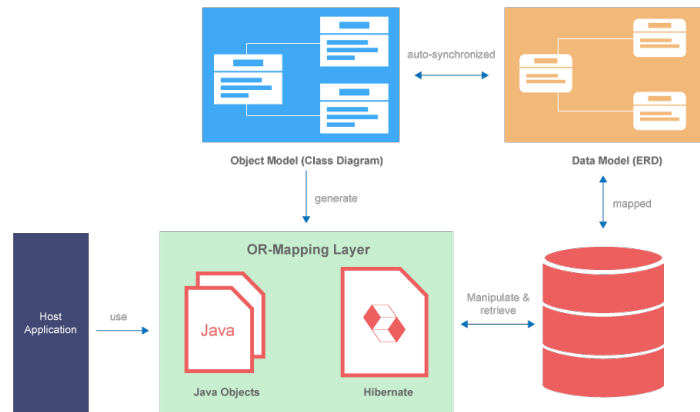


Fig. 3: Relational to Object Oriented Data Mapping in Java Applications using Hibernate [7]

Most *ORMs* follow a similar architecture and provide similar services [8]. We will use the *Hibernate ORM* architecture as the reference architecture for our purposes in this study [7, 9]. *Hibernate* is a longstanding open-source solution with a well understood architecture. The framework interfaces between the host application and database management system to provide automatic conversion of data between the two models. Fig. 3 shows how this is typically done from a high-level perspective. *Hibernate* starts with the generation of the object model, which can then be mapped automatically to a relational data model of the same data domain. The data access layer of the host application is built on top of *Hibernate ORM framework*.

Next, we can examine *Hibernate* more at a more granular level as seen in Fig. 4. The *ORM* provides session management, transaction management, connection management and drivers for several database providers.

ORMs are very powerful systems with many complexities. As well due to their close proximity to the database of an application, the security considerations are of utmost importance when testing their code bases. As we will see, it is not only the code that needs to be examined, but as well the supporting files containing metadata, configurations, and build information.

1.3 Team Contribution

As a team we tried our best to make it possible for each team member to contribute equal amounts to this project. To break up the workload, it was decided that the 3 tools used would be divided among the 6 group members, grouping us into 3 pairs. Each pair was responsible for learning, applying, analyzing the results, and outputting raw data for their designated tool. We then contributed equally to the preparation and delivery of our presentation. We also contributed equally to the writing of our final report.

2 Selecting Open-Source Projects

Project	Category	Language
ORM	Hibernate [10]	Java
	OpenJPA [11]	Java
	EbeanORM [12]	Java
	PonyORM [13]	Python
	SQLAlchemy [14]	Python
Web Framework	Spring Boot	Java
	Struts	Java
	Django	Python
	Flask	Python
	Fastapi	Python
Android Project	Nextcloud	Java
	MPAndroidChart	Java
	NewPipe	Java, Kotlin
	Firefox	Kotlin
	Ivy Wallet	Kotlin
Web Server	Apache httpd	C
	Nginx	C
	Mongoose	C

Table 1: Open Source Projects Researched

We started by brainstorming potential project categories and found projects in each. The project categories we collected projects for were *web frameworks*, *ORMs*, *android projects*, and *web servers*. We then narrowed down our potential project categories to *web frameworks* and *ORMs* since the projects are written in *Java* and *Python* which all team members are familiar with. Ultimately, we decided to select *ORMs* as our project category since the project sizes are smaller and they're more specialized than *web frameworks*.

The researched and selected projects are listed in [Table 1](#) and [Table 2](#).

Project	Source Lines	GitHub Stars	Language
Hibernate [10]	1356004	5700	Java
OpenJPA [11]	643700	128	Java
EbeanORM [12]	674438	1400	Java
PonyORM [13]	28294	3500	Python
SQLAlchemy [14]	400463	8700	Python

Table 2: Open Source Projects Selected to be Analyzed

3 Selecting Detection Tools

3.1 Researched Tools

Type	Tool	Language
Static Code Analyzer	Snyk	Java, Python
	Find Bugs	Java
	PMD	Java, Python
	Coverity Scan	Java, Python
	PyLint	Python
	PyFlakes	Python
	Code Climate	Java, Python
	FitNesse	Java, Python
	Sonarcloud	Java, Python
	LLM	Java, Python
Test Case Generator	Fuzzinator	Java, Python
	Evosuite	Java
Record & Replay	Selenium	Java, Python
Dynamic Invariant Detection	Daikon	Java

Table 3: Bug Detection Tools Researched

3.2 Selection Process

Similar to the project selection process, tool selection began with listing out possible tools that could be used. This list was focused on tools that work on more than one programming language, specifically *Java* and *Python*. The tools were grouped based on whether they were *static*, *dynamic*, *record & replay*, or *test case generation*.

From the researched tools we narrowed the list down to trying 6 of the tools based on the fact that they all supported both *Java* and *Python*. We tried *Snyk*, *SonarCloud*, *Coverity Scan*, *Code Climate*, *Fuzzinator*, and *PMD*. The 3 tools that were then selected to perform the full analysis were *Snyk*, *SonarCloud*, and *Coverity Scan*. These tools were selected because we felt that they were the most accessible to use, had good collaboration tools, and had the best dashboards.

3.3 Selected Tools

3.3.1 Snyk

Snyk is a browser-based developer security platform that allows application and cloud developers to ensure security within their applications [15]. It prioritizes developers to see and fix every vulnerability found in their system in an easy-to-use and quick manner while maintaining integrity by using industry standards for computer security testing.

Snyk allows for integration directly into the *CI/CD pipeline* of any project to scan to find any security vulnerabilities and license issues found as well as an API to allow developers to enable automation for any workflow they desire. These features help defer *Snyk* from other tools on the market and with its easy adoption for any company or individual, it allows for the world to have better security at ease.

Unlike the other static analysis testing tools stated in this report, *Snyk* is only security-based and does not find abnormalities, code smells, or basic issues which will be highlighted in the following sections. This makes *Snyk* a valid option to consider when working on projects as it checks for issues other testing tools may miss.

3.3.2 SonarCloud

SonarCloud is a cloud-based static analysis tool by *SonarSource* SA as part of their *Sonar Solution*. The tool focuses on clean code analysis after the build phase of the source code [16, 17]. Moreover, it works by integrating itself into an applications *CI/CD pipeline* as see in [Fig. 5](#). This allows for an extra layer of quality assurance before newly developed code is merged with production code. The *Sonar Solution* is used by millions of developers across 400K+ organizations [16].

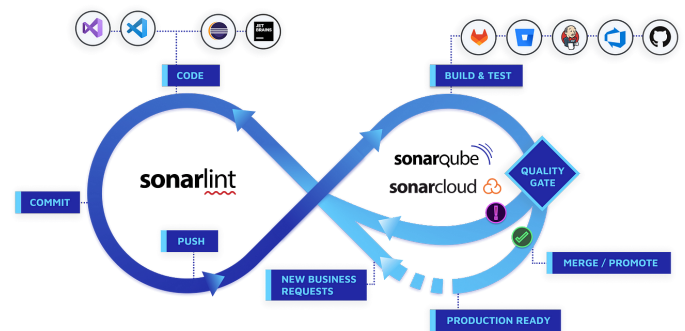


Fig. 5: SonarSource Clean Code Solution Workflow [17]

3.3.3 Coverity

Coverity Scan is a static code analysis service provided by *Synopsys* for open-source projects. *Synopsys* created *Coverity* in 2006 as part of a 3 year contract with the U.S. Department of Homeland Security. The goal of *Coverity Scan* was to improve software quality and security within open-source projects.

Coverity Scan is free to use for open-source projects. *Synopsys* specifies that their implementation of static code analysis can follow all possible execution paths, allowing for complete path coverage. This allows *Coverity Scan* to locate bugs that are a result of a specific path [18].

4 Applying Detection Tools

To break up the workload, it was decided that the 3 tools would be divided amongst the 6 group members, grouping us into 3 pairs. Within these pairs, one person would apply their assigned tool to the *Java* projects, and the other to the *Python* projects.

4.1 Snyk

4.1.1 Process

To analyze the chosen projects with *Snyk* we used the *GitHub* repository method. We signed in with our *GitHub* account and added the respective *repositories* we needed. These *repositories* must be added to our account (either owned or forked).

After you will be prompted to select the *repositories* you wish to test, proceeding with the automated testing thereafter which takes approximately five minutes to complete testing and generate reports.

Once reports are generated, they can be looked at through the *Snyk* website through their interactive UI, which shows plots, timelines, and estimations for bug fixes. This same data can be exported as raw data either using *RestAPI* or by downloading the data as a *.csv* file.

4.1.2 Difficulties

In terms of completing the process, *Snyk* has a very intuitive interface and experience which makes retrieving basic data regarding issues simple, in which it organizes each issue by severity (**C = Critical**, **H = High Issue**, **M = Medium Issue**, **L = Low Issue**).

Difficulties did arise when attempting to retrieve the raw data from *Snyk* where accessing the respective *.csv* files for each *ORM* was locked behind the paid version of the service. This required us to start a free trial of the paid version of the software to access it. In addition, to properly format the raw data retrieved, we had to reformat the data to be consistent in the project (4.1.3).

4.1.3 Raw Data Retrieval

When retrieving the raw data from the *.csv* files, they included information that we used in our refined data, but also unusable data. Data that was used includes the *issue severity* and *issue type*. Data that was unused includes *severity score*, *project URL* (which did not work), and *Snyk codes* (in reference to their documentation).

To format the *.csv* files to directly correlate to the raw data of the other tools used in this project, we performed an automated reading of the features from *Snyk's* website using knowledge from HTML. This then helped us retrieve missing data such as the *path* of each issue, *line number* in reference to its location in code, and each issue's respective *description*.

4.2 SonarCloud

4.2.1 Process

To analyze the chosen projects with *SonarCloud*, we first created *SonarCloud* accounts and created an organization for our team so we could collaborate on the same projects. We then created projects for each of the *ORM* projects within the *SonarCloud* dashboard.

To run the analysis on each project we chose to do the manual method. Using *Linux*, we cloned each *ORM's* open-source repository locally. The *Sonar Scanner* tool was then downloaded and some environment variables needed to be setup. We then executed the tool for each project and the analysis started. This process was different depending on if the project was in *Java* or *Python* since the *Java* projects have build automation tools like *Maven* or *Gradle*. The commands are as follows:

- For Java Maven: `mvn verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar -Dsonar.projectKey=<project key> -DskipTests -Dsonar.scm.disabled=true`
- For Java Gradle: `./gradlew sonar`
- For Python: `sonar-scanner -Dsonar.organization=<organization> -Dsonar.projectKey=<project key> -Dsonar.sources=. -Dsonar.host.url=https://sonarcloud.io`

When the *Sonar Scanner* tool was finished the results were automatically uploaded to the *SonarCloud* dashboard.

4.2.2 Difficulties

The main difficulty we had with *SonarCloud* was during the analysis process for *Java* projects. Since *Java* projects need to be built using *Maven* or *Gradle* they took some troubleshooting to successfully build. The first issue we had was due to using *Java 8* and was fixed by updating to use *Java 17*. Another issue we had was some of the project's dependencies needed updating. After a bunch of troubleshooting we were able to get all the *Java* projects to build but it wouldn't be easy to do if you don't have good troubleshooting skills. Its worth noting that the actual developers of these projects may have an easier time running the analysis process since they should already have the build automation working in their development environments.

4.2.3 Raw Data Retrieval

The API to retrieve data is found at: [Issues API](#) [19]. The data is available as JSON files. The schema is very detailed and comprehensive. There are four API parameters to consider when building a request. First, *componentKeys*, for this field the value must be the project key as found in the project information page. Second, *ps*, is the page size and can be used to select exactly how many results are wanted in the request. Third, *impactSoftwareQualities*, this is

a comma-delimited list of any of **RELIABILITY, SECURITY, MAINTAINABILITY**. We used the former two as the latter is concerned with thousands of style suggestions rather than true bugs. Finally, `access_token`, the account holders API key.

4.3 Coverity Scan

4.3.1 Process

To analyze the chosen projects with *Coverity Scan*, we first forked the *GitHub* repositories of the projects and created personal copies of them. We then signed into *Coverity Scan* with the same *GitHub* account, went to the 'My Dashboard' section, and clicked on the 'Add a project' button.

From there, for each project, we registered a new project by clicking on the 'Register My GitHub Project' button, clicking on 'Reload repository list from GitHub' to reload the list of projects, and selecting one of the listed forked repositories.

A form was then shown where the 'Repository Name', 'Repository URL', and 'Homepage URL' sections were already filled out and we entered our 'Role' as Contributor, entered the programming language used by the project that we wanted to scan into the 'Language' field, and entered the open-source license used by the project in the 'License' field. *Coverity Scan* will check if the project meets the open-source project requirement, and bug analysis results will not be available until they confirm this.

After submitting the form we were taken to a new section for the project. From there we then had to use *Coverity Scan*'s Self-Build tool to create and submit a build of the project for the *Coverity Scan* tool to analyze. We first clicked on the 'Submit Build' button on the project's overview page, which took us to a page for uploading a build, and then we clicked the 'Download Coverity Scan Self-Build' button. This took us to a page with instructions for creating and uploading a build specific to the language entered for the project where we could also download the *Coverity Scan* build tool.

Once the *Coverity Scan* build tool is installed, this tool can be used to locally build the projects. The steps to build a *Java Project* for *Coverity Scan* analysis are as follows:

- For Gradle Projects: **`cov-build -dir cov-int gradlew build`**
- For Maven Projects: **`cov-build -dir cov-int mvn compile`**

The steps to build a *Python Project* for *Coverity Scan* analysis are as follows:

- Run the following in a Linux terminal:
 - Change directory to build directory: **`cd <build/directory>`**
 - Create cov-int build: **`cov-build -dir cov-int --no-command -fs-capture-search <path/to/source/code>`**
 - Zip cov-int build: **`zip -r <projectname>.zip cov-int`**
- Upload build zip file to the build upload page

Once the project builds are successful the *cov-int* folder can be zipped, and uploaded to the *Coverity Scan* dashboard for analysis. The project will enter a queue for analysis, and an email will be sent once the bug analysis is completed. This may take up to 48 hours.

4.3.2 Difficulties

There was initially some confusion about how to run *Coverity* on the chosen projects as the only version of *Coverity* available to the

general public is *Coverity Scan* which is an online service instead of the typical downloadable tool or add-on. *Coverity Scan* does have downloadable tools but these are only available to employees of companies with a license for them and members of academic institutions that are registered with Synopse's academic program. After learning all of this, we decided to try *Coverity Scan*.

In looking into using *Coverity Scan*, we were uncertain about whether the service would allow us to analyze our chosen projects. The first point of uncertainty was that the tool only permits contributors and maintainers or owners of a project to submit it for analysis. The other point was that after analyzing the project, the details of the analysis and the defects or bugs detected can not be accessed until the entered role of the user to the project and the project's license are reviewed, which could take 1-2 business days. These factors accumulated into the potential issues of time wasted waiting for our projects to be approved and the potential for the tool to be unusable if our project submissions were denied. Ultimately, we went ahead and submitted the projects with our role entered as contributors, since finding bugs and reporting them qualifies us for that role, and the projects were thankfully approved within a day.

For the Java Projects specifically, we ran into some difficulties due to the nature of *Coverity Scan* requiring a project build to be submitted. Our project selection initially included a project that used *Apache Ant* as opposed to *Maven* or *Gradle*. We were unable to successfully build this project locally, meaning it could not be analyzed. As a result of these issues, we removed that project from the project selection list. Additionally, there was quite a bit of time spent to successfully build the three Java projects, given that they all required different *Java*, *Maven*, and *Gradle* versions. One important feature of *Coverity Scan* is that it only analyzes the files that are built when the *cov-build* command is used. This means that if there was a previous partially successful build, the most recent use of *cov-build* would only build the remaining files. This occurred with the first run of *Hibernate*, but it was not discovered until the analysis results were returned, and we noticed it only scanned 6000 lines, finding only 22 bugs. *Hibernate* was later re-analyzed, to ensure the full code base was cleaned and then built, giving proper results. However, it took over 24 hours to get the results back the second time.

4.3.3 Raw Data Retrieval

Coverity Scan provides the bugs in a row and column format, with an option to filter the bugs, as well as only include features/columns of interest. These columns can be exported as a CSV file. The default columns include information regarding the type of bug, as well as which file, function, and line number it occurs at. There is also the option to manually add data to each bug, such as if it's been investigated and found to be a false positive. Features like these are useful for projects where the scan is performed periodically, so the bug does not re-appear each time.

To conform with the formatting across all bug detection tools, we shuffled the columns to align with an agreed upon format. This was done within a Jupyter Notebook using *Python* and *Pandas*.

5 Evaluation Results

5.1 Snyk

5.1.1 Results

Project	# of Bugs	Severity			
		Low	Medium	High	Critical
Hibernate	77	75	2	0	0
OpenJPA	61	17	22	20	2
EbeanORM	37	8	19	10	0
PonyORM	14	12	1	1	0
SQLAlchemy	61	56	6	0	0

Table 4: Snyk Bug Detection Results

Distribution of issues by type in Snyk Detection Across All ORM Libraries

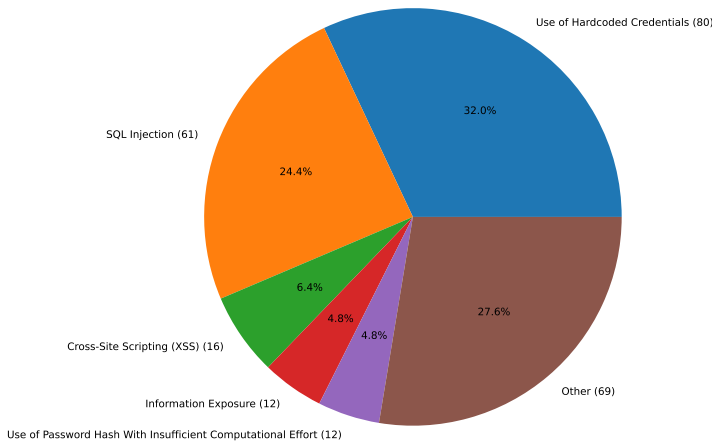


Fig. 6: Snyk Results By Issue Type

When looking at issues detected by *Snyk*, *Use of Hardcoded Credentials* and *SQL Injection* issues were the most common, which were considered *low severity* by *Snyk*. These issues also made up most of the false positives we determined in our results. This can be seen in the pie chart in Fig. 6.

When looking at issues detected across all frameworks with *Snyk*, *Hibernate*, *OpenJPA* and *SQLAlchemy* are the *ORMs* with the most issues and while *Ebean* and *PonyORM* have fewer issues. This correlates with the size of the *ORMs* respectively as large projects have more issues. This can be seen in the pie chart in Fig. 7.

5.1.2 Detected Bug Samples

Snyk's Project Code Analysis Tool highlights how each bug/vulnerability is presented after the code is tested, with each issue having its section respectively. The code analysis presents the user with the issue/vulnerability's *severity* (denoted by the letter and colour), *issue's name*, *severity score* (modified version of the *CVSS Framework Version 3.1* [20]), the *code segment & line*, and *reasoning for issue*.

Snyk's Project Code Analysis Report highlights how each project/repositories *issues of each severity, unique vulnerabilities*, a

Distribution of Issues Detected in Each ORM Library by Snyk

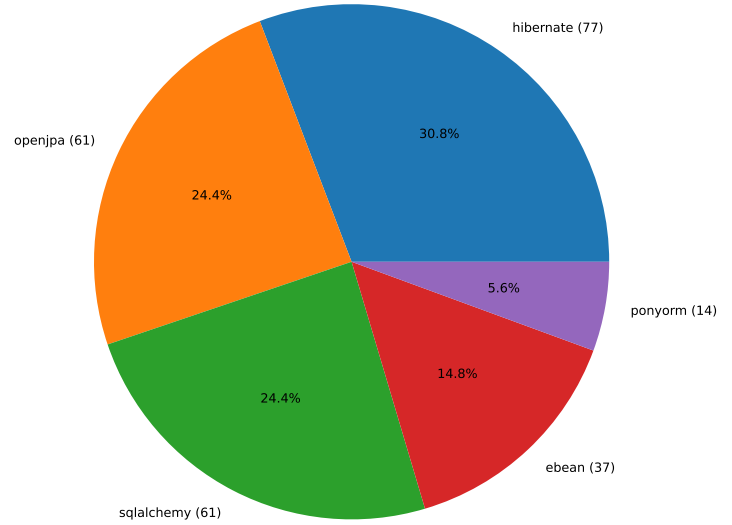


Fig. 7: Snyk Issues By ORM



Fig. 8: Viewing Bug Using Snyk Project Code Analysis

small list for each issue, and the ability to export the data as a *.csv* files (used to retrieve raw data).

When analyzing the *Snyk report* and *raw data* given, we were able to manually determine what were false positives in the issues found. The false positives were decided based on factors such as *location in code*, *legitimacy of threat*, and *exploitability*. For example, license issues are classified as false positives because they are not software bugs and contain no known exploits. For hard-coded credentials, the location of the issue helped classify these issues because if the issue is in the test component of the project then it is known that the credentials are only used for testing and have no real impact. Finally, the legitimacy of the threat was also used this was gauged based on how readily exploitable an issue was if it contained no known exploits then it was classified as a false positive because

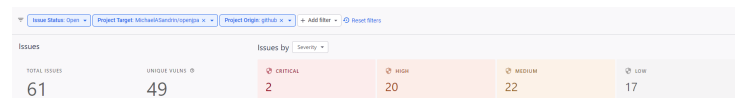


Fig. 9: Viewing Code Analysis Report in Snyk

there are no known exploits for this issue

Using *Snyk's* calculated sample size and manually determined the number of *false positives* (6.2.2). The number of false positives derived from all *five ORMs* using *Snyk* totalled to **13**, and when calculated with a sample size of **22**, the *false positive rate* of *Snyk* is **59.09%**.

5.2 SonarCloud

5.2.1 Results

SonarCloud finds a large number of issues of varying severity across each framework. This is summarized in Table 5. The type of issues is almost all bug and very few are classified as vulnerabilities as shown in Fig. 10 by the uneven proportion of bugs to vulnerabilities. Note that this distinction is *SonarCloud's* own classification of the bugs.

Project	# of Bugs	Severity		
		Low	Medium	High
Hibernate	457	172	251	34
OpenJPA	487	35	399	53
EbeanORM	202	59	114	29
PonyORM	54	2	40	12
SQLAlchemy	333	17	208	108

Table 5: Sonarcloud Bug Detection Results

Distribution of Issues Detected in Each ORM Library By SonarCloud

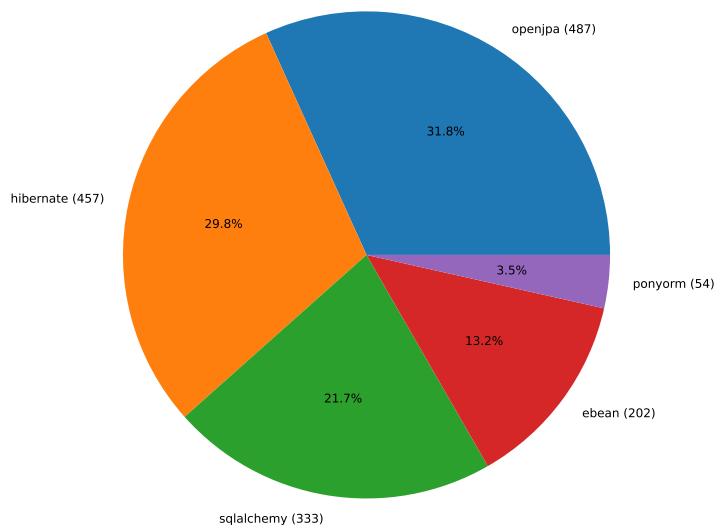


Fig. 11: SonarCloud Issues By ORM

5.2.2 Detected Bug Samples

SonarCloud's dashboard lists all the found bugs/issues from the analysis. Each bug is given a severity (Low, Medium, High) to help prioritize fixing. The reasoning for why *SonarCloud* thinks the found bug is an issue and suggestions on how to fix it are also given. The bug found in Fig. 11 showcases an issue in *SQLAlchemy* where there seems to be a redundant if statement.

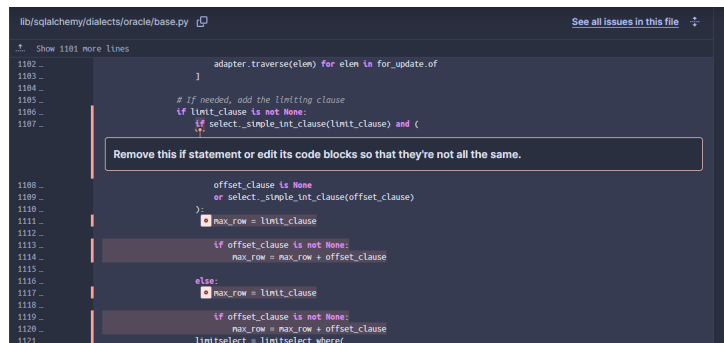


Fig. 12: Viewing Bug Using SonarCloud Dashboard

Distribution of issues by type in SonarCloud Detection Across All ORM Libraries

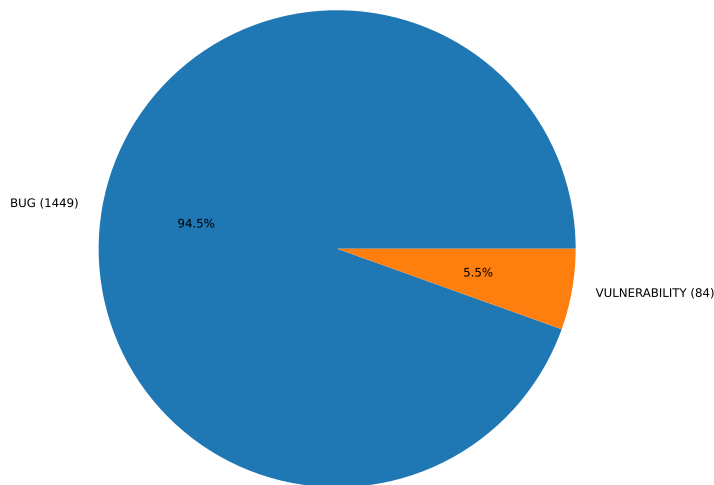


Fig. 10: SonarCloud Results By Issue Type

When looking at issues detected across all frameworks, *OpenJPA*, *Hibernate* and *SQLAlchemy* are the largest categories and while *PonyORM* has a very small total number of issues. This is in line with what is expected as the former three are the largest code bases and *PonyORM* is very lightweight. This can be seen in the pie chart in

5.3 Coverity

5.3.1 Results

Coverity Scan was able to find a total of 981 bugs across the five projects we applied it to. The most common severity of bugs found was Medium, with High severity bugs only be found for the Java projects. The *Coverity Scan* results are summarized within Table 6. Additionally, the *Coverity Scan* results found that the most common type of issue detected were quality issues, followed by security issues, as seen in Fig. 13. More specifically, common bugs it found included concurrency data access, API usage errors, resource leaks, null pointer deference's, class hierarchy inconsistencies, bad

Project	# of Bugs	Severity		
		Low	Medium	High
Hibernate	345	15	298	32
OpenJPA	353	14	290	49
EbeanORM	185	91	92	2
PonyORM	38	0	38	0
SQLAlchemy	60	20	40	0

Table 6: Coverity Scan bug detection results.

Distribution of issues by type in Coverity Detection Across All ORM Libraries

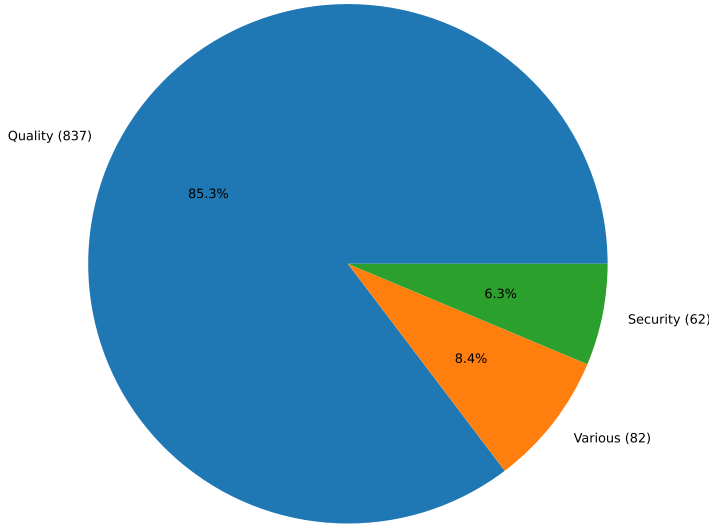


Fig. 13: Coverity Results By Issue Type

use of null-like, logically dead code, and copy-paste errors. *Coverity Scan* was run across 5 projects, with the results summarized in Fig. 14. Over 50% of the bugs found by *Coverity Scan* were in *OpenJPA*. Overall, it found 92.7% of the total bugs within the 3 *Java* projects, and only 7.7% of the total bugs within the 2 *Python* projects.

5.3.2 Detected Bug Samples

The bugs caught by *Coverity Scan* were evaluated with the assistance of the tool's analysis services which provide a generated table of defects, information on the type of defect and what triggered the detection, and embedded annotations in the relevant sections of the code on where each defect was detected and what path the tool took to detect each defect. This information was compared to the relevant code logic, structure, and comments to evaluate false positives.

For instance, *Coverity Scan* detected many "Bad use of null-like value" defects in the *PonyORM* project's source code. As shown in Fig. 15, clicking on one of these defects in the table provides thorough information about the defect. This information includes how the defect was detected for property access in which an attempt to access the property of an object with a null-like value was made. Annotations are also provided with the source code indicating the specific line of issue, and the variable values chosen and path taken to reach the issue.

Distribution of Issues Detected in Each ORM Library by Coverity

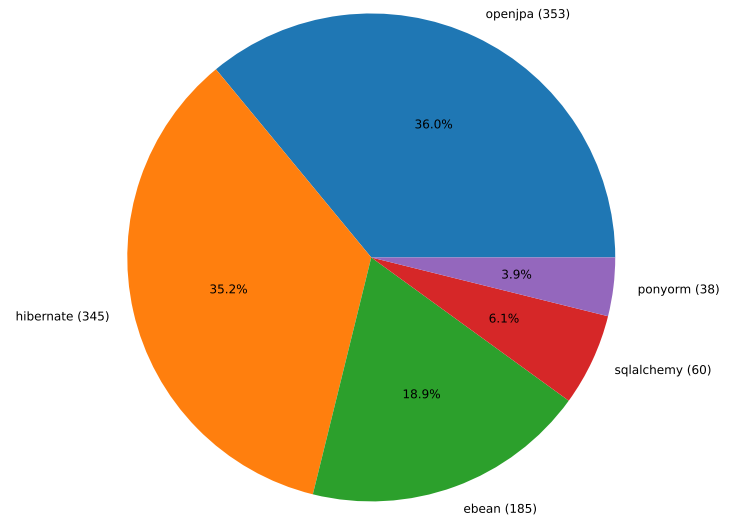


Fig. 14: Coverity Issued By ORM

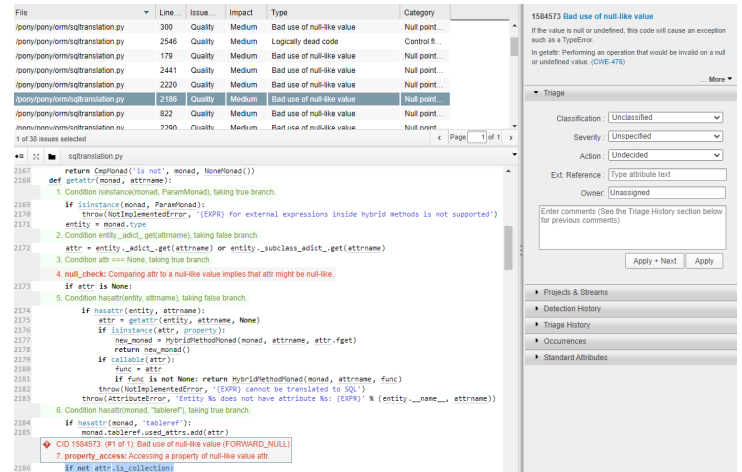


Fig. 15: Coverity Scan Analysis Services with PonyORM Defect

The structure, logic, and comments in the displayed section of code were then manually examined and compared to the tool's provided information to determine whether the bug was a false positive. In the case of the property access defect, it was determined to be a false positive because the path leading to the property access of the null-like object would terminate before reaching the code line of issue. An early check is made for whether the object is not None and custom error throwing functions are triggered if false. *Coverity Scan* can't follow this logic so it does not understand that the path leading to the issue is not reachable.

6 Discussion

6.1 Pros & Cons

After using all three tools for these open-source *ORM frameworks*, a list of pros and cons for each tool can be found in Tables 7, 8, and 9. These assertions are based on usage of the tools along

with specifications and the exact results produced.

Pros	Cons
1. Free (For limited monthly use) 2. Easy Compatibility with various different languages 3. Bugs are given a score based on industry consensus and standards 4. Web-based dashboard easily accessible and shareable 5. Vast support and documentation	1. Only targets security bugs/vulnerabilities 2. Costly when used often for projects 3. Finds many Low Severity false positives 4. Requires paid version to retrieve raw data

Table 7: Pros & Cons of Snyk

Pros	Cons
1. Free (For open-source projects) 2. Compatibility with various different languages 3. Provides information on priority and severity of bugs 4. Web based dashboard easily accessible and shareable 5. Full explanations for found bugs/issues with possible solutions & estimated fix time 6. CI/CD pipeline support	1. Harder to run on Java projects 2. Can only export raw data through REST API 3. Manual scan, did not show coverage of code

Table 8: Pros & Cons of SonarCloud

6.2 Tool Comparison

6.2.1 Bug Density

When comparing tools, the importance of highlighting *bug density* as a metric is key to deciding which tool is more effective on average. It provides a qualitative measure that gives a general understanding of a software's quality, with less dense software being higher in quality. When more bugs are found by the static analysis tools used in the project, it highlights the effectiveness of each tool. In this project, we calculated the *bug density* as the *number of bugs per 1k lines of code*. This comparison can be seen in Table 10.

6.2.2 False Positives

To determine the false positive rate of each tool we evaluated a sample of the bugs detected by each tool and determined if each bug was a valid bug or not.

To select the sample size for each tool we used a sample size calculator [21] which determines the appropriate sample size given

Pros	Cons
1. Thorough defect and vulnerability results 2. Extensive bug documentation 3. User interface to navigate through defects • Annotates source code with line of issue and path required for failure • Paths annotated across methods/classes and include values that reach pathway 4. Can configure components and functions to decrease false positive rate 5. CI/CD pipeline support	1. Requires a project build for analysis. • Building Java source code with Coverity Scan's Self-Build tool requires the use of a build automation tool such as Maven or Gradle • Difficult to set up analysis for Java projects 2. Relying on Synopsys to analyze code and return results to you • Build enters a queue of projects to be analyzed • Results may take up to 48 hours to receive 3. Coverity Scan results appear to be vary between analysis runs

Table 9: Pros & Cons of Coverity Scan

	Snyk	SonarCloud	Coverity
Hibernate	0.134	0.78	0.59
OpenJPA	0.223	2.13	1.35
Ebean ORM	0.198	1.64	1.56
PonyORM	0.483	1.86	1.23
SQLAlchemy	0.150	0.64	0.11

Table 10: Bug Density (Number of Bugs Found per 1000 LOC)

the population size and confidence level. We then randomly selected samples from the bugs detected by each tool using a *Python script*.

Each bug was manually evaluated by a team member and was classified as either a true positive or false positive. We used our own intuition and searched each project's issue tracking system for similar bugs in order to make a decision. Its important to note that we are not active developers of these projects so our classification on whether the bug is valid or not could be incorrect. Then we calculated a false positive rate for each tool which can be seen in Table 11.

$$toolFalsePositiveRate = \frac{falsePositiveAmount}{toolSampleSize} \quad (1)$$

7 Suggested Tool & Conclusion

This project examined three static code analysis tools *SonarCloud*, *Coverity Scan*, and *Snyk* for bug detection within open-source

	Bugs Sampled	False Positive Rate (%)
Snyk	22	59.09%
SonarCloud	24	20.83%
Coverity	24	12.5%

Table 11: False Positive Rates for Each Tool (From sampling)

Java and Python Object Relational Mappers (ORMs). Each tool demonstrated unique strengths and weaknesses. *Snyk*, focuses on security vulnerabilities, but lacked bug detection beyond security and had a high false positive rate. *SonarCloud*, was the most comprehensive in its analysis, has a reasonable false positive rate, and offers insightful bug explanations, but it was challenging to run on Java projects. *Coverity Scan* delivered good results with extensive documentation, has a reasonable false positive rate, but suffered from lengthy analysis run-times and inconsistencies between runs.

Considering the overall performance, *SonarCloud* emerges as our suggested tool for bug detection in *ORMs*. Its user friendly interface, detailed explanations, and ability to identify various types of issues, makes it the most accessible tool to improve code quality. While the challenges with Java projects should be considered, if integrated with a CI/CD pipeline these issues should be one offs.

References

- [1] [Online]. Available: <https://github.com/Hashir-Jamil/static-analysis-tools-comparison>
- [2] W. Landi, "Undecidability of static analysis," *ACM Lett. Program. Lang. Syst.*, vol. 1, no. 4, p. 323–337, dec 1992. [Online]. Available: <https://doi.org/10.1145/161494.161501>
- [3] "How static analysis works." [Online]. Available: https://verifysoft.com/en_grammatech_how_static_analysis_works.html
- [4] H. Prähofer, F. Angerer, R. Ramler, H. Lacheiner, and F. Grillenberger, "Opportunities and challenges of static code analysis of iec 61131-3 programs," in *Proceedings of 2012 IEEE 17th International Conference on Emerging Technologies Factory Automation (ETFA 2012)*, 2012, pp. 1–8. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/6489535>
- [5] "Static code analysis in continuous integration and continuous delivery (ci/cd)." [Online]. Available: <https://www.mathworks.com/products/polyspace/static-analysis-notes/continuous-integration-continuous-delivery.html>
- [6] C. Ireland, D. Bowers, M. Newton, and K. Waugh, "A classification of object-relational impedance mismatch," in *2009 First International Conference on Advances in Databases, Knowledge, and Data Applications*, 2009, pp. 36–43. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/5071809>
- [7] "https://www.visual-paradigm.com/features/hibernate-orm-tools/." [Online]. Available: <https://www.javatpoint.com/hibernate-architecture>
- [8] A. Torres, R. Galante, M. S. Pimenta, and A. J. B. Martins, "Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design," *Information and Software Technology*, vol. 82, pp. 1–18, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584916301859>
- [9] "Hibernate architecture." [Online]. Available: <https://www.javatpoint.com/hibernate-architecture>
- [10] "hibernate-orm." [Online]. Available: <https://github.com/hibernate/hibernate-orm>
- [11] "openjpa." [Online]. Available: <https://github.com/apache/openjpa>
- [12] "ebean." [Online]. Available: <https://github.com/ebean-orm/ebean>
- [13] "pony." [Online]. Available: <https://github.com/ponyorm/pony>
- [14] "sqlalchemy." [Online]. Available: <https://github.com/sqlalchemy/sqlalchemy/>
- [15] "What is snyk?" [Online]. Available: <https://snyk.io/product/>
- [16] "Sonar." [Online]. Available: <https://www.sonarsource.com/>
- [17] "Sonarcloud documentation." [Online]. Available: <https://docs.sonarsource.com/sonarcloud/>
- [18] "Coverity frequently asked questions." [Online]. Available: <https://scan.coverity.com/faq>
- [19] "Web api." [Online]. Available: https://sonarcloud.io/web_api/
- [20] "Common vulnerability scoring system: Specification document." [Online]. Available: <https://www.first.org/cvss/specification-document>
- [21] "Sample size calculator." [Online]. Available: <https://www.abs.gov.au/websitedbs/D3310114.nsf/home/Sample+Size+Calculator?opendocument>